

Desenvolvimento Web com .NET

Aula 6 – Detalhar, Editar e Remover

Prof. Me. Lucas Teodoro dos Santos

UNIP

16 de Setembro de 2026

Roteiro da Aula

- 1 Diagrama Listar Médicos
- 2 Diagrama Inserir Médico
- 3 Adicionar Ações na Listagem
- 4 Detalhar Médico
- 5 Editar Médico
- 6 Remover Médico
- 7 Diagrama das Principais Classes

Diagrama de sequência – Listar Médicos

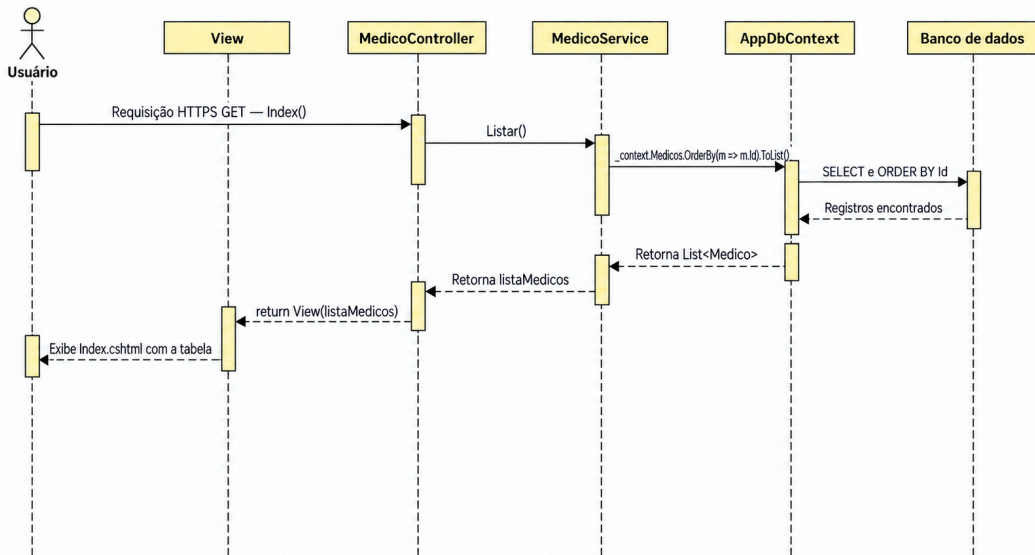
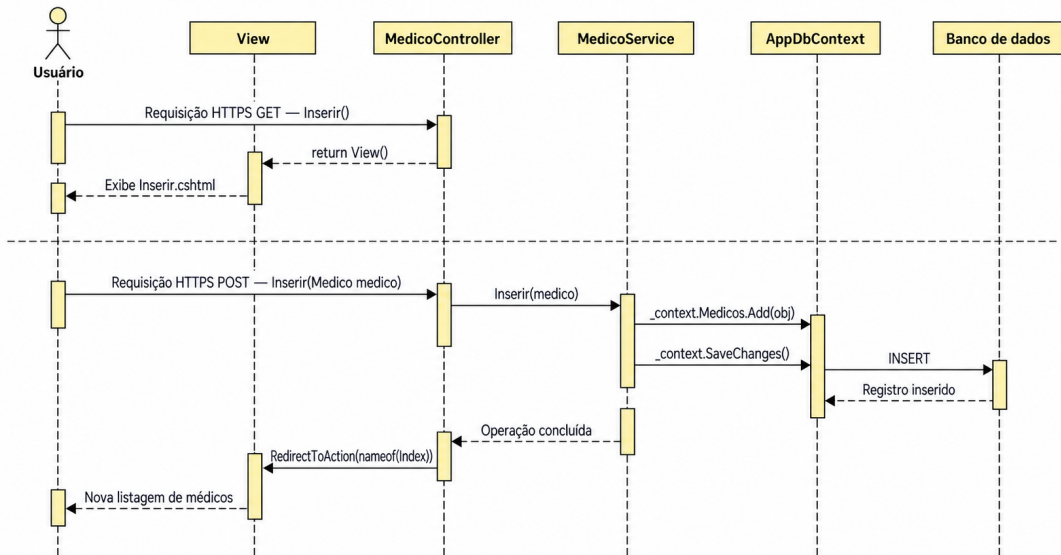


Diagrama de sequência – Inserir Médico



Até o momento, a aplicação realiza uma comunicação **síncrona** com o banco de dados.

Isso significa que, ao solicitar uma operação ao banco, a aplicação aguarda o resultado antes de seguir para a próxima instrução.

Posteriormente na disciplina, será apresentada a comunicação assíncrona, utilizando `async` e `await`.

Adicionando ações à listagem de médicos

Na página `Index`, cada médico apresentado na tabela terá botões para acessar as funcionalidades disponíveis.

Os botões recebem o identificador do médico selecionado por meio de `asp-route-id`.

As ações adicionadas serão:

- **Detalhar:** Apresenta todas as informações do médico;
- **Editar:** Permite alterar as informações do médico;
- **Remover:** Abre a tela de confirmação da exclusão.

Botões de ação na página Index

```
1      @foreach(var m in Model)
2      {
3          <tr>
4              <td>@m.Id</td>
5              <td>@m.Nome</td>
6              <td>@m.Crm</td>
7              <td>@m.Especialidade</td>
8
9              <td class="acoes">
10                 <a asp-action="Detalhar" asp-route-id="@m.Id" class="btn btn-outline-warning">
11                     Detalhar</a>
12                 <a asp-action="Editar" asp-route-id="@m.Id" class="btn btn-outline-info">Editar
13                     </a>
14                 <a asp-action="Remover" asp-route-id="@m.Id" class="btn btn-outline-danger">
15                     Remover</a>
16             </td>
17         </tr>
18     }
19 }
```

Ajustando a coluna de ações

Foi criado um estilo para a coluna que contém os botões de ação da tabela.

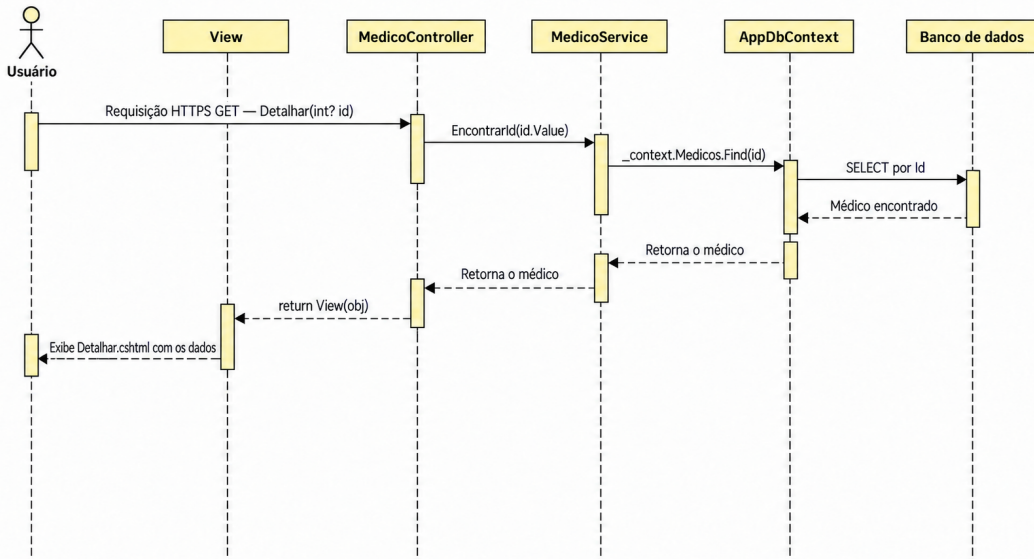
A classe `acoes` evita que os botões sejam quebrados em mais de uma linha e faz com que a coluna ocupe apenas o espaço necessário.

O código deve ser adicionado no arquivo:

`wwwroot/css/site.css`

```
1 .acoes {  
2     width: 1%;  
3     white-space: nowrap;  
4 }
```


Diagrama de sequência – Detalhar Médico



A funcionalidade Detalhar permite visualizar todas as informações de um médico selecionado na página de listagem.

Para implementá-la, serão realizados os seguintes passos:

- 1 Criar o método `EncontrarId()` no `MedicoService`;
- 2 Criar a ação `Detalhar` no `MedicoController`;
- 3 Criar a view `Detalhar` para apresentar os dados do médico.

O método de busca por identificador também será reutilizado nas funcionalidades de editar e remover.

Buscar médico por identificador

```
1 public Medico? EncontrarId(int id)
2 {
3     return _context.Medicos.Find(id);
4 }
```

Ação Detalhar no MedicoController

A ação recebe o identificador do médico selecionado, realiza sua busca e envia o objeto encontrado para a view Detalhar.

```
1 public IActionResult Detalhar(int? id)
2 {
3     if (id == null)
4         return NotFound();
5
6     var obj = _medicoService.EncontrarId(id.Value);
7
8     if (obj == null)
9         return NotFound();
10
11     return View(obj);
12 }
```

View Detalhar – 1/2

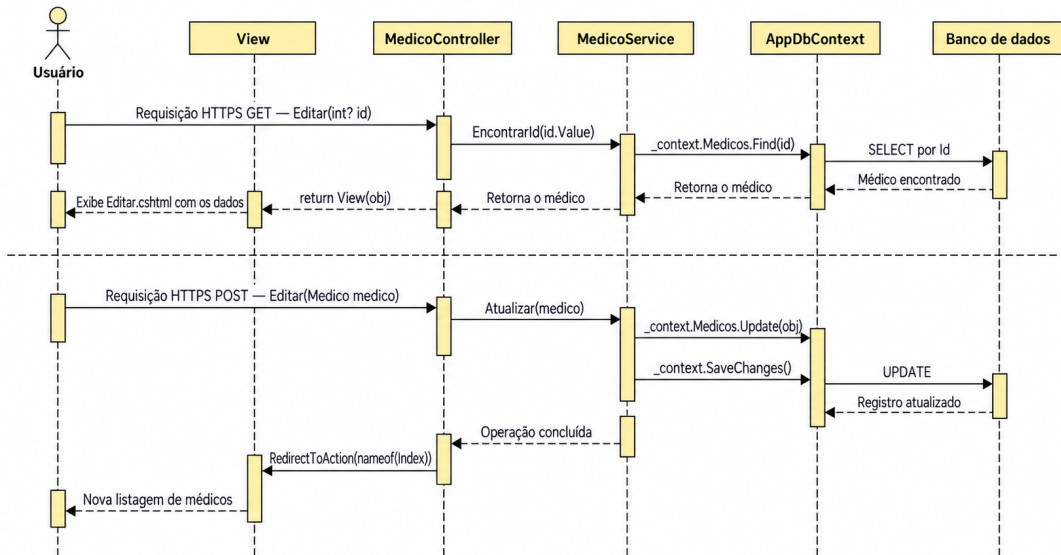
A view recebe o médico selecionado e apresenta seus dados. Ao final, são disponibilizados botões para editar o registro ou voltar à listagem.

```
1      @model Agendamento.Models.Medico
2
3      @{
4          ViewData["Title"] = "Detalhes Medico";
5      }
6
7      <h1>Detalhes</h1>
8
9      <div>
10         <hr />
11         <dl class="row">
12             <dt class="col-sm-2">
13                 Nome
14             </dt>
15             <dd class="col-sm-10">
16                 @Model.Nome
17             </dd>
```

View Detalhar – 2/2

```
1      <dt class="col-sm-2">
2          Crm
3      </dt>
4      <dd class="col-sm-10">
5          @Model.Crm
6      </dd>
7
8      <dt class="col-sm-2">
9          Especialidade
10     </dt>
11     <dd class="col-sm-10">
12         @Model.Especialidade
13     </dd>
14 </dl>
15
16 <a asp-action="Editar" asp-route-id="@Model.Id" class="btn btn-outline-secondary">
17     Editar</a>
18 <a asp-action="Index" class="btn btn-outline-info">Voltar</a>
19
20 <hr />
</div>
```

Diagrama de sequência – Editar Médico



Implementação – Editar Médico

A funcionalidade **Editar** permite alterar as informações de um médico já cadastrado no banco de dados.

Para realizar a edição, serão utilizadas duas ações no controller:

- Uma ação com GET, responsável por buscar o médico e apresentar seus dados no formulário;
- Uma ação com POST, responsável por receber os dados alterados e atualizar o registro.

Para implementar essa funcionalidade, serão realizados os seguintes passos:

- 1 Criar o método `Atualizar()` no `MedicoService`;
- 2 Criar a ação `Editar` com GET;
- 3 Criar a view `Editar`;
- 4 Criar a ação `Editar` com POST.

Classe MedicoService – Atualizar Médico

O método recebe o objeto com os dados alterados e solicita ao Entity Framework Core a atualização do registro.

O identificador presente no objeto permite que o Entity Framework Core reconheça qual registro deve ser atualizado.

```
1 public void Atualizar(Medico obj)
2 {
3     _context.Medicos.Update(obj);
4     _context.SaveChanges();
5 }
```

Ação Editar com GET

A ação com GET recebe o identificador do médico selecionado, busca o registro e envia o objeto encontrado para a view.

As verificações impedem que a aplicação tente apresentar um médico quando o identificador não foi informado ou o registro não foi encontrado.

```
1 public IActionResult Editar(int? id)
2 {
3     if (id == null)
4         return NotFound();
5
6     var obj = _medicoService.EncontrarId(id.Value);
7
8     if (obj == null)
9         return NotFound();
10
11     return View(obj);
12 }
```

A view `Editar` recebe o médico localizado pela ação com GET e apresenta seus dados em um formulário.

Como os campos utilizam `asp-for`, seus valores são preenchidos automaticamente com os dados do objeto recebido.

O identificador é armazenado em um campo oculto, pois ele não deve ser alterado pelo usuário, mas precisa ser enviado junto com o formulário.

Ao salvar, os dados são enviados para a ação `Editar` com POST.

View Editor – 1/2

```
1 @model Agendamento.Models.Medico
2
3 @{
4     ViewData["Title"] = "Editar Medico";
5 }
6
7 <h4>Medico</h4>
8
9 <hr />
10 <div class="row">
11     <div class="col-md-4">
12         <form asp-action="Editar">
13             <input type="hidden" asp-for="Id" />
14             <div class="mb-3">
15                 <label asp-for="Nome" class="control-label"></label>
16                 <input asp-for="Nome" class="form-control"/>
17             </div>
18             <div class="mb-3">
19                 <label asp-for="Crm" class="control-label"></label>
20                 <input asp-for="Crm" class="form-control" />
21             </div>
```

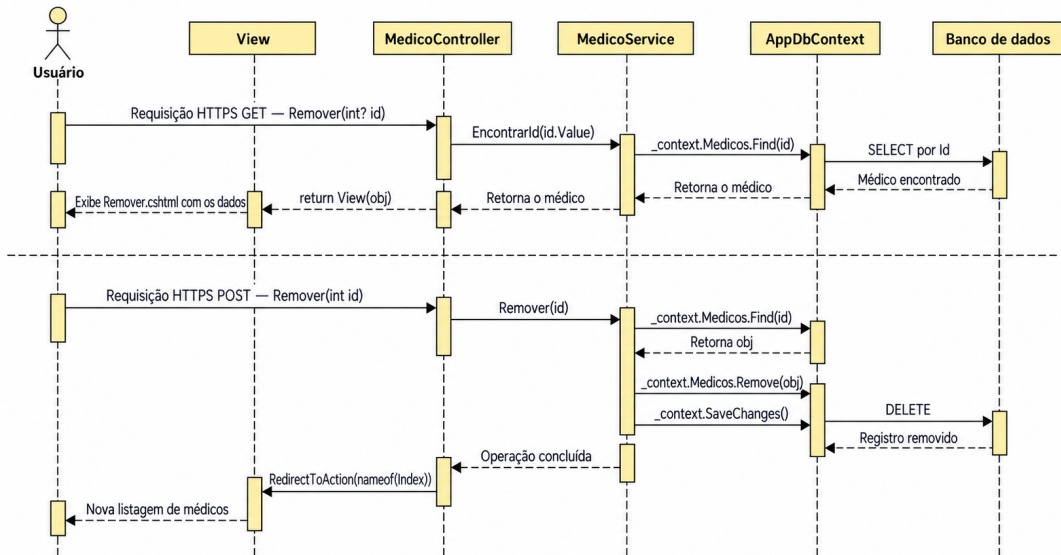
```
1      <div class="mb-3">
2          <label asp-for="Especialidade" class="control-label"></label>
3          <input asp-for="Especialidade" class="form-control" />
4      </div>
5      <div class="mt-3">
6          <input type="submit" value="Salvar" class="btn btn-outline-success"/>
7          <a asp-action="Index" class="btn btn-outline-info">Voltar</a>
8      </div>
9  </form>
10 </div>
11 </div>
```

Ação Editar com POST

A ação com POST recebe o objeto preenchido com os valores enviados pelo formulário. O método Atualizar() é chamado e as alterações são gravadas no banco de dados. Após a atualização, o usuário retorna para a página de listagem.

```
1      [HttpPost]
2      [ValidateAntiForgeryToken]
3      public IActionResult Editar(Medico obj)
4      {
5          _medicoService.Atualizar(obj);
6          return RedirectToAction(nameof(Index));
7      }
```

Diagrama de sequência – Remover Médico



Implementação – Remover Médico

A remoção de um médico será realizada após a confirmação do usuário.

Primeiramente, a aplicação apresenta os dados do médico selecionado em uma tela de confirmação. Após a confirmação, o registro é removido do banco de dados.

Para implementar essa funcionalidade, serão realizados os seguintes passos:

- 1 Reutilizar o método `EncontrarId()` e criar o método `Remover()` no `MedicoService`;
- 2 Criar a ação `Remover` com GET no controller;
- 3 Criar a view `Remover` para apresentar os dados e solicitar a confirmação;
- 4 Criar a ação `Remover` com POST no controller.

Remover médico

```
1 public void Remover(int id)
2 {
3     var obj = _context.Medicos.Find(id);
4
5     if (obj == null)
6     {
7         return;
8     }
9
10    _context.Medicos.Remove(obj);
11    _context.SaveChanges();
12 }
```

Ação Remover com GET

Esta ação recebe o identificador do médico selecionado e prepara a tela de confirmação da remoção.

Antes de enviar o médico para a view, são realizadas duas verificações:

- Verifica se o identificador foi informado;
- Verifica se existe um médico com esse identificador.

Se o médico for localizado, seus dados são enviados para a view Remove.

```
1 public IActionResult Remover(int? id)
2 {
3     if (id == null)
4         return NotFound();
5
6     var obj = _medicoService.EncontrarId(id.Value);
7
8     if (obj == null)
9         return NotFound();
10
11     return View(obj);
12 }
```

A view `Remover` possui uma estrutura semelhante à view `Detalhar`, pois também recebe o médico selecionado e apresenta seus dados.

A principal diferença é que, além de apresentar as informações, esta view possui um formulário para confirmar a exclusão do registro.

Ao selecionar o botão `Remover`, o identificador do médico é enviado para a ação `Remover` com `POST`, que realiza a exclusão no banco de dados.

View Remover – 1/2

```
1 @model Agendamento.Models.Medico
2
3 @{ ViewData["Title"] = "Remover Medico"; }
4
5 <h1>Remover</h1>
6
7 <h3>Tem certeza de que deseja remover este medico?</h3>
8
9 <div>
10     <hr />
11     <dl class="row">
12         <dt class="col-sm-2">
13             Nome
14         </dt>
15         <dd class="col-sm-10">
16             @Model.Nome
17         </dd>
18
19         <dt class="col-sm-2">
20             Crm
21         </dt>
```

View Remover – 2/2

```
1      <dd class="col-sm-10">
2          @Model.Crm
3      </dd>
4
5      <dt class="col-sm-2">
6          Especialidade
7      </dt>
8      <dd class="col-sm-10">
9          @Model.Especialidade
10     </dd>
11 </dl>
12
13 <form asp-action="Remover">
14     <input type="hidden" asp-for="Id" />
15     <input type="submit" value="Remover" class="btn btn-outline-danger" />
16     <a asp-action="Index" class="btn btn-outline-info">Voltar</a>
17 </form>
18
19 <hr />
20 </div>
```

Ação Remover com POST

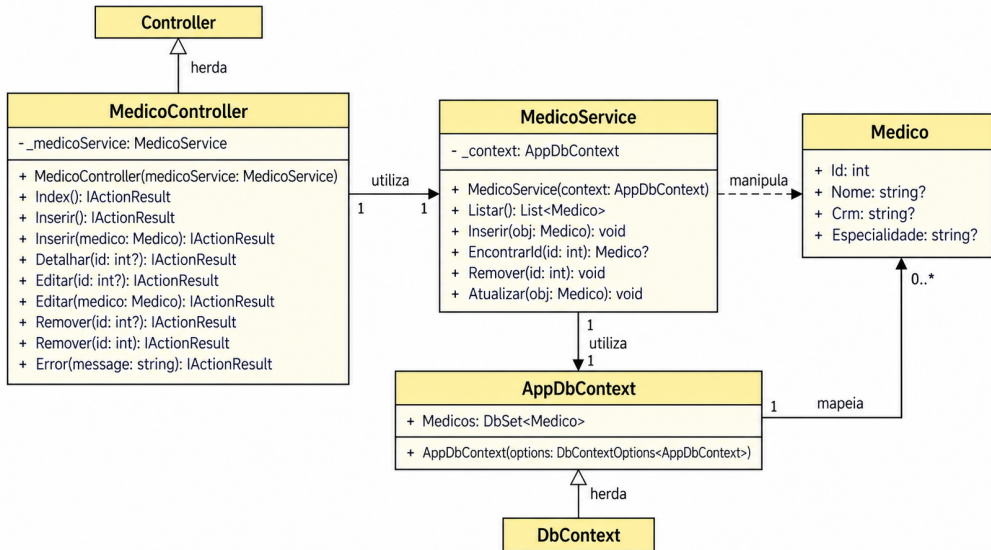
Após a confirmação do usuário na view Remover, esta ação recebe o identificador do médico selecionado.

A ação chama o método Remover() do MedicoService, que exclui o registro do banco de dados.

Após a exclusão, o usuário é redirecionado para a ação Index, que apresenta novamente a lista de médicos.

```
1 [HttpPost]
2 [ValidateAntiForgeryToken]
3 public IActionResult Remover(int id)
4 {
5     _medicoService.Remover(id);
6     return RedirectToAction(nameof(Index));
7 }
```

Diagrama das Principais Classes



Próximos passos para evolução do sistema

O sistema desenvolvido apresenta uma implementação introdutória das operações de CRUD: listar, inserir, detalhar, editar e remover médicos.

Em uma aplicação real, ainda são necessárias outras etapas para tornar o sistema mais seguro, confiável e completo:

- Validação dos campos informados pelo usuário;
- Tratamento de exceções e apresentação de mensagens adequadas;
- Confirmação e feedback após cada operação realizada;
- Validação de regras de negócio;
- Garantia de dados únicos, como o CRM do médico;
- Autenticação e controle de acesso dos usuários;
- Testes das funcionalidades;
- Implementação das entidades de pacientes e agendamentos.

O CRUD representa a base da aplicação e poderá ser ampliado progressivamente nas próximas etapas do desenvolvimento.

Resumo Geral da Aula

Nesta aula, o sistema de agendamento foi ampliado com as demais operações do CRUD de médicos.

Foram abordados os seguintes pontos:

- Diagramas de sequência das funcionalidades de listar, inserir, detalhar e remover médicos;
- Inclusão dos botões de ação na página **Index**;
- Implementação do recurso **Detalhes**;
- Implementação do recurso **Editar**;
- Implementação do recurso **Remover**;
- Diagrama das classes principais da aplicação.

A aplicação agora permite listar, inserir, detalhar, editar e remover registros de médicos no banco de dados.

Obrigado!

Dúvidas?